Documentation

The Java™ Tutorials
Hide TOC
Creating and Using Extensions
Installed Extensions
Download Extensions
Understanding Extension Class Loading
Creating Extensible Applications
Home Page > The Extension Mechanism > Creating and Using Extensions
« Previous • Trail • Next »
The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent release
See JDK Release Notes for information about new features, enhancements, and removed or deprecated option

Creating Extensible Applications
The following topics are covered:

Introduction
Dictionary Service Example
Running the DictionaryServiceDemo Sample
Compiling and Running the DictionaryServiceDemo Sample
Understanding the DictionaryServiceDemo Sample
Define the Service Provider Interface
Define the Service That Retrieves the Service Provider Implementations
The Singleton Design Pattern
Implement the Service Provider
Register Service Providers
Create a Client That Uses the Service and Service Providers
Package the Service Providers, the Service, and the Service Client in JAR Files
Packaging Service Providers in JAR Files
Packaging the Dictionary SPI and Dictionary Service in a JAR File
Packaging the Client in a JAR File
Run the Client
The ServiceLoader Class
Limitations of the ServiceLoader API
Summary
Introduction
An extensible application is one that you can extend without modifying its original code base. You can e

This section describes how to create applications with extensible services, which enable you or others t

One example of an extensible application is a word processor that allows the end user to add a new dicti

The following are terms and definitions important to understand extensible applications:

Service
A set of programming interfaces and classes that provide access to some specific application functionali
Service provider interface (SPI)
The set of public interfaces and abstract classes that a service defines. The SPI defines the classes an
Service Provider
Implements the SPI. An application with extensible services enable you, vendors, and customers to add se
Dictionary Service Example
Consider how you might design a dictionary service in a word processor or editor. One way is to define a

Although the word-processor developer would most likely provide a basic, general dictionary with the ori

The DictionaryServiceDemo sample shows you how to implement a Dictionary service, create Dictionary serv

build.xml
DictionaryDemo
build.xml
build
dist
DictionaryDemo.jar
src
dictionary
DictionaryDemo.java
DictionaryServiceProvider
build.xml
build
dist
DictionaryServiceProvider.jar
src
dictionary
DictionaryService.java
spi
Dictionary.java
ExtendedDictionary
build.xml

build
dist
ExtendedDictionary.jar
src
dictionary
ExtendedDictionary.java
META-INF
services
dictionary.spi.Dictionary
GeneralDictionary
build.xml
build
dist
GeneralDictionary.jar
src
dictionary
GeneralDictionary.java
META-INF
services
dictionary.spi.Dictionary
Note: The build directories contain the compiled class files of the Java source files contained in the s

Running the DictionaryServiceDemo Sample
Because the zip file DictionaryServiceDemo.zip contains compiled class files, you can unzip this file to

Download and unzip the sample code: Download and unzip the file DictionaryServiceDemo.zip to your comput

Change the current directory to C:\DictionaryServiceDemo\DictionaryDemo and follow the step Run the Clie

Compiling and Running the DictionaryServiceDemo Sample
The DictionaryServiceDemo sample includes Apache Ant build files, which are all named build.xml. The fol

Install Apache Ant: Go to the following link to download and install Apache Ant:

http://ant.apache.org/

Ensure that the directory that contains the Apache Ant executable file is in your PATH environment varia

Download and unzip the sample code: Download and unzip the file DictionaryServiceDemo.zip to your comput

Compile the code: Change the current directory to C:\DictionaryServiceDemo and run the following comman

ant compile-all
This command compiles the source code in the src directories contained in the directories DictionaryDemo

Package the compiled Java files into JAR files: Ensure the current directory is C:\DictionaryServiceDemo

ant jar
This command creates the following JAR files:

DictionaryDemo/dist/DictionaryDemo.jar
DictionaryServiceProvider/dist/DictionaryServiceProvider.jar
GeneralDictionary/dist/GeneralDictionary.jar
ExtendedDictionary/dist/ExtendedDictionary.jar
Run the sample: Ensure that the directory that contains the java executable is in your PATH environment

Change the current directory to C:\DictionaryServiceDemo\DictionaryDemo and run the following command:

ant run
The sample prints the following:

book: a set of written or printed pages, usually bound with a protective cover
editor: a person who edits
xml: a document standard often used in web services, among other things
REST: an architecture style for creating, reading, updating, and deleting data that attempts to use the

Understanding the DictionaryServiceDemo Sample
The following steps show you how to re-create the contents of the file DictionaryServiceDemo.zip. These

1. Define the Service Provider Interface
The DictionaryServiceDemo sample defines one SPI, the Dictionary.java interface. It contains only one me

```
package dictionary.spi;

public interface Dictionary {
    public String getDefinition(String word);
}
```

The sample stores the compiled class file in the directory DictionaryServiceProvider/build.

2. Define the Service That Retrieves the Service Provider Implementations

The DictionaryService.java class loads and accesses available Dictionary service providers on behalf of

```
package dictionary;

import dictionary.spi.Dictionary;
import java.util.Iterator;
import java.util.ServiceConfigurationError;
import java.util.ServiceLoader;

public class DictionaryService {

    private static DictionaryService service;
    private ServiceLoader<Dictionary> loader;

    private DictionaryService() {
        loader = ServiceLoader.load(Dictionary.class);
    }

    public static synchronized DictionaryService getInstance() {
        if (service == null) {
            service = new DictionaryService();
        }
        return service;
    }


    public String getDefinition(String word) {
        String definition = null;

        try {
            Iterator<Dictionary> dictionaries = loader.iterator();
            while (definition == null && dictionaries.hasNext()) {
                Dictionary d = dictionaries.next();
                definition = d.getDefinition(word);
            }
        } catch (ServiceConfigurationError serviceError) {
            definition = null;
            serviceError.printStackTrace();

        }
        return definition;
    }
}
```

The sample stores the compiled class file in the directory DictionaryServiceProvider/build.

The DictionaryService class implements the singleton design pattern. This means that only a single insta

The DictionaryService class is the dictionary service client's entry point to using any installed Dictio

The dictionary service uses the ServiceLoader.load method to find the target class. The SPI is defined b

However, an overloaded version of this method enables you to specify custom class loaders if you wish. T

After a loader for this class exists, you can use its iterator method to access and use each provider th

The DictionaryDemo.java class uses this service. To use the service, the application obtains a Dictionar

The Singleton Design Pattern
A design pattern is a general solution to a common problem in software design. The idea is that the solu

For example, the DictionaryService class implements the singleton pattern as follows:

Declares the DictionaryService constructor as private, which prevents all other classes, except Dictiona
Defines the DictionaryService member variable service as static, which ensures only one instance of Dict
Defines the method getInstance, which enables other classes controlled access to the DictionaryService m
3. Implement the Service Provider
To provide this service, you must create a Dictionary.java implementation. To keep things simple, create

The following code shows an implementation of the Dictionary SPI, the GeneralDictionary.java class. Noti

```
package dictionary;

import dictionary.spi.Dictionary;
import java.util.SortedMap;
import java.util.TreeMap;

public class GeneralDictionary implements Dictionary {

    private SortedMap<String, String> map;
```

```
    public GeneralDictionary() {
        map = new TreeMap<String, String>();
        map.put(
            "book",
            "a set of written or printed pages, usually bound with " +
                "a protective cover");
        map.put(
            "editor",
            "a person who edits");
    }

    @Override
    public String getDefinition(String word) {
        return map.get(word);
    }

}
```
The sample stores the compiled class file in the directory GeneralDictionary/build. Note: You must compi

The GeneralDictionary provider for this example defines just two words: book and editor. Obviously, a mo

To demonstrate how multiple providers can implement the same SPI, the following code shows yet another p

```
package dictionary;

import dictionary.spi.Dictionary;
import java.util.SortedMap;
import java.util.TreeMap;

public class ExtendedDictionary implements Dictionary {

        private SortedMap<String, String> map;

    public ExtendedDictionary() {
        map = new TreeMap<String, String>();
        map.put(
            "xml",
            "a document standard often used in web services, among other " +
                "things");
        map.put(
            "REST",
            "an architecture style for creating, reading, updating, " +
                "and deleting data that attempts to use the common " +
                "vocabulary of the HTTP protocol; Representational State " +
                "Transfer");
    }

    @Override
    public String getDefinition(String word) {
        return map.get(word);
    }

}
```
The sample stores the compiled class file in the directory ExtendedDictionary/build. Note: You must comp

It is easy to imagine customers using a complete set of Dictionary providers for their own special needs

4. Register Service Providers
To register your service provider, you create a provider configuration file, which is stored in the META

The provider configuration file contains the fully qualified class names of your service providers, one

For example, to register the service provider GeneralDictionary create a text file named dictionary.spi.

```
dictionary.GeneralDictionary
```
Similarly, to register the service provider ExtendedDictionary create a text file named dictionary.spi.D

```
dictionary.ExtendedDictionary
```
5. Create a Client That Uses the Service and Service Providers
Because developing a full word-processor application is a significant undertaking, this tutorial provide

The following is the DictionaryDemo sample. It requests a definition of the target word from the Diction

```
package dictionary;

import dictionary.DictionaryService;

public class DictionaryDemo {
  public static void main(String[] args) {
```

```
    DictionaryService dictionary = DictionaryService.getInstance();
    System.out.println(DictionaryDemo.lookup(dictionary, "book"));
    System.out.println(DictionaryDemo.lookup(dictionary, "editor"));
    System.out.println(DictionaryDemo.lookup(dictionary, "xml"));
    System.out.println(DictionaryDemo.lookup(dictionary, "REST"));
  }

  public static String lookup(DictionaryService dictionary, String word) {
    String outputString = word + ": ";
    String definition = dictionary.getDefinition(word);
    if (definition == null) {
      return outputString + "Cannot find definition for this word.";
    } else {
      return outputString + definition;
    }
  }
}
```

The sample stores the compiled class file in the directory DictionaryDemo/build. Note: You must compile

6. Package the Service Providers, the Service, and the Service Client in JAR Files

See the lesson Packaging Programs in JAR Files for information about how to create JAR files.

Packaging Service Providers in JAR Files

To package the GeneralDictionary service provider, create a JAR file named GeneralDictionary/dist/Genera

```
META-INF
services
dictionary.spi.Dictionary
dictionary
GeneralDictionary.class
```

Similarly, to package the ExtendedDictionary service provider, create a JAR file named ExtendedDictionar

```
META-INF
services
dictionary.spi.Dictionary
dictionary
ExtendedDictionary.class
```

Note that the provider configuration file must be in the directory META-INF/services in the JAR file.

Packaging the Dictionary SPI and Dictionary Service in a JAR File

Create a JAR file named DictionaryServiceProvider/dist/DictionaryServiceProvider.jar that contains the f

```
dictionary
DictionaryService.class
spi
Dictionary.class
```

Packaging the Client in a JAR File

Create a JAR file named DictionaryDemo/dist/DictionaryDemo.jar that contains the following file:

```
dictionary
DictionaryDemo.class
```

7. Run the Client

The following command runs the DictionaryDemo sample with the GeneralDictionary service provider:

Linux and Solaris:

```
java -Djava.ext.dirs=../DictionaryServiceProvider/dist:../GeneralDictionary/dist -cp dist/DictionaryDemo
```

Windows:

```
java -Djava.ext.dirs=..\DictionaryServiceProvider\dist;..\GeneralDictionary\dist -cp dist\DictionaryDemo
```

When using this command, the following is assumed:

The current directory is DictionaryDemo.
The following JAR files exist:
DictionaryDemo/dist/DictionaryDemo.jar: Contains the DictionaryDemo class
DictionaryServiceProvider/dist/DictionaryServiceProvider.jar: Contains the Dictionary SPI and the Dictio
GeneralDictionary/dist/GeneralDictionary.jar: Contains the GeneralDictionary service provider and config
The command prints the following:

```
book: a set of written or printed pages, usually bound with a protective cover
editor: a person who edits
xml: Cannot find definition for this word.
REST: Cannot find definition for this word.
```

Suppose you run the following command and ExtendedDictionary/dist/ExtendedDictionary.jar exists:

Linux and Solaris:

```
java -Djava.ext.dirs=../DictionaryServiceProvider/dist:../ExtendedDictionary/dist -cp dist/DictionaryDem
```

Windows:

```
java -Djava.ext.dirs=..\DictionaryServiceProvider\dist;..\ExtendedDictionary\dist -cp dist\DictionaryDem

The command prints the following:

book: Cannot find definition for this word.
editor: Cannot find definition for this word.
xml: a document standard often used in web services, among other things
REST: an architecture style for creating, reading, updating, and deleting data that attempts to use the
```

The ServiceLoader Class

The java.util.ServiceLoader class helps you find, load, and use service providers. It searches for servi

The ServiceLoader class is final, which means that you cannot make it a subclass or override its loading

From the perspective of the ServiceLoader class, all services have a single type, which is usually a sin

Providers are located and instantiated on demand. A service loader maintains a cache of the providers th

To create a loader for a specific class, provide the class itself to the load or loadInstalled method. Y

The loadInstalled method searches the runtime environment's extension directory of installed runtime pro

Limitations of the ServiceLoader API

The ServiceLoader API is useful, but it has limitations. For example, it is impossible to derive a class

The public ServiceLoader API is available in Java SE 6. Although the loader service existed as early as

Summary

Extensible applications provide service points that can be extended by service providers. The easiest wa

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Cookie-præferencerAd Choices